

COMPTE-RENDU SAÉ : STATION MÉTÉO



BUT1 GEII TD_C
Années: 2025/2026
Par :
SARR Mansour
SAICHE Massil
NDYAYE Mafall
GUEDJALI Riyad

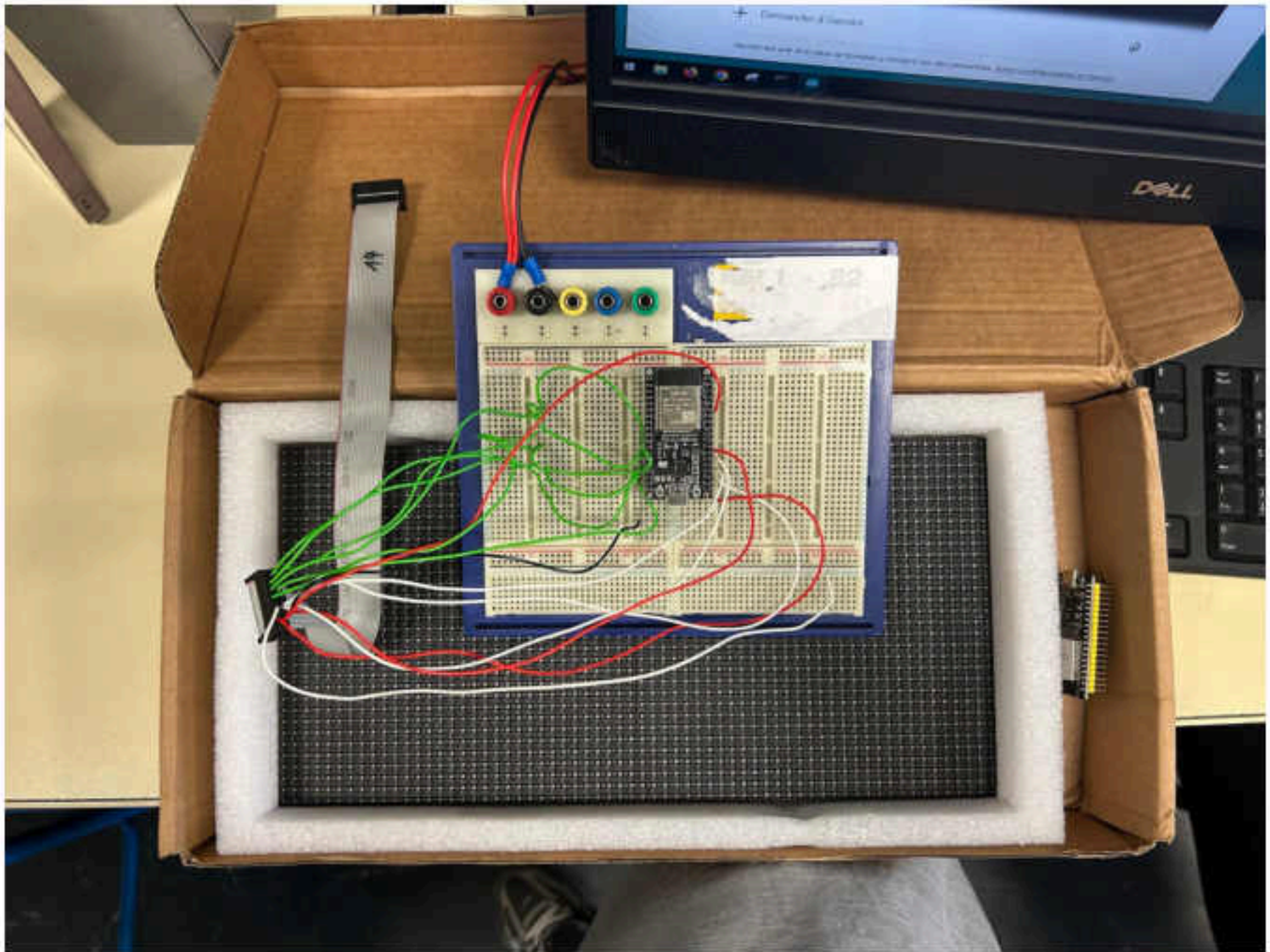
Projet encadré par :
M. JEAN MARIE Henri

SOMMAIRE :

Introduction Général.....	3
Configuration et Maîtrise Bas Niveau (Séquence 1).....	4
1.1 Configuration matérielle et logicielle	
1.2 Réponses aux Questions d'Exercices (Initialisation OE, Chronogramme, envoyer 64 Pixels et fixer Adresse)	
1.3 Réponses aux Questions d'Auto-Contrôle (Logique inversée, Temporisation et Multiplexage partagé)	
Exploitation de Bibliothèques et Synchronisation NTP (Séquence 2).....	7
2.1 Apport du contrôleur DMA face au rafraîchissement manuel	
2.2 Réponses aux Questions d'Auto-Contrôle (Encombrement textuel, Mécanisme NTP et Espacement des zones)	
Acquisition I2C et Rétroaction Lumineuse (Séquence 3).....	9
3.1 Architecture du Bus I2C et Scanner de périphériques	
3.2 Algorithme d'adaptation lumineuse automatique	
3.3 Justification de l'utilisation de millis() face à delay()	
Conclusion générale.....	13

Introduction Générale

Ce projet de SAE consiste à concevoir et programmer une **Station Météo Connectée** à l'aide d'un microcontrôleur ESP32 et d'une matrice LED RGB 32x64 fonctionnant sous le protocole HUB75. L'objectif est d'afficher de manière fluide, structurée et éco-responsable l'heure synchronisée via le réseau, ainsi que des grandeurs physiques (température, humidité, luminosité) acquises en temps réel via un bus de communication I2C.



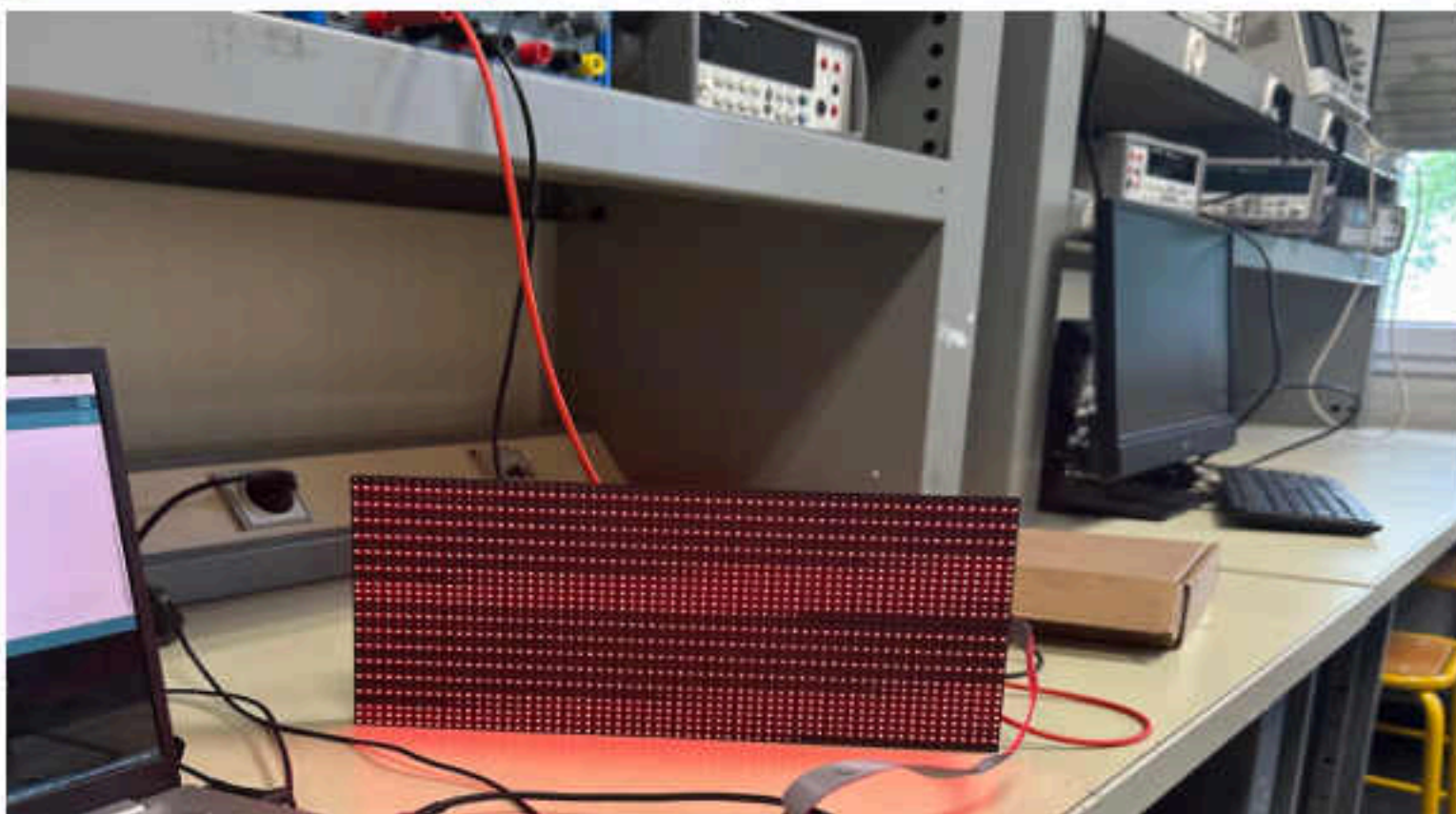
1. CONFIGURATION ET MAÎTRISE BAS NIVEAU (Séquence 1)



1.1 Configuration matérielle et logicielle

Pour programmer la carte ESP32 (interface USB-Série CP2102) , l'environnement Arduino IDE a été configuré en ajoutant le lien JSON d'Espressif dans les préférences , puis en sélectionnant le type de carte **ESP32 Dev Module**. La vitesse de téléversement a été fixée à **921600 baud** pour optimiser les temps de programmation.

Rappel de sécurité : Le panneau LED consomme un courant important. Il a impérativement été alimenté par un bloc externe 5V régulé. L'alimenter directement via le port USB de l'ESP 32 est strictement proscrit sous peine d'endommager la carte.



1.2 Réponses aux Questions d'Exercices (Séquence 1)

Exercice 1 : Initialisation de la broche OE

Le signal OE (*Output Enable*) est actif **BAS**. Pour s'assurer que le panneau reste éteint au démarrage (état inactif du panneau), il faut lui appliquer un état électrique **HAUT (HIGH)**. La ligne de code correspondante dans le `setup()` est :

C++

```
digitalWrite(OE, HIGH); // [cite: 86].
```

Exercice 2 : Chronogramme et fonction `envoyer 64 Pixels`

1. **Analyse du basculement** : Il faut basculer le signal LAT (Latch) pendant que le signal OE est **inactif (éteint, donc à l'état HIGH)**.
2. **Pourquoi ?** : Cela évite le phénomène de *ghosting* (lignes floues ou rémanence visuelle) , car si OE restait actif pendant le verrouillage, les pixels transitoires seraient brièvement visibles à l'écran pendant leur déplacement dans les registres de décalage.

Code complété pour l'impulsion LATCH :

C++

```
void envoyer64Pixels (bool r, bool g, bool b) { // [cite: 85]
// 1. Désactivation de l'affichage pendant le transfert série
digitalWrite(OE, HIGH); // [cite: 86]

// 2. Boucle série pour charger les 64 colonnes
for (int i=0; i<64; i++) { // [cite: 88]
    digitalWrite(R1, r); // [cite: 90]
    digitalWrite(G1, g); // [cite: 91]
    digitalWrite(B1, b); // [cite: 93]

    digitalWrite(CLK, HIGH); // [cite: 96]
    delayMicroseconds(1); // [cite: 97]
    digitalWrite(CLK, LOW); // [cite: 98]
    delayMicroseconds(1); // [cite: 99]
}

// 3. Impulsion HAUT puis BAS sur la broche LAT (Complété) [cite: 101]
digitalWrite(LAT, HIGH); // Transfert vers les verrous de sortie [cite: 83]
delayMicroseconds(1);
digitalWrite(LAT, LOW); // Fin de l'impulsion [cite: 83]
}
```

Exercice 3 : Explication de la fonction **fixé Adresse**

La fonction **fixe Adresse(int ligne)** permet de sélectionner la ligne double à allumer (parmi les 16 interconnexions physiques du multiplexage 1/16). Elle utilise des opérateurs de décalage de bits (**>>**) et un masquage binaire (**& 1**) pour extraire l'état (0 ou 1) de chaque bit du nombre entier **ligne**, puis l'applique sur les broches du bus ABCD (A = bit 0, B = bit 1, C = bit 2, D = bit 3).

1.3 Réponses aux Questions d'Auto-Contrôle (Séquence 1)

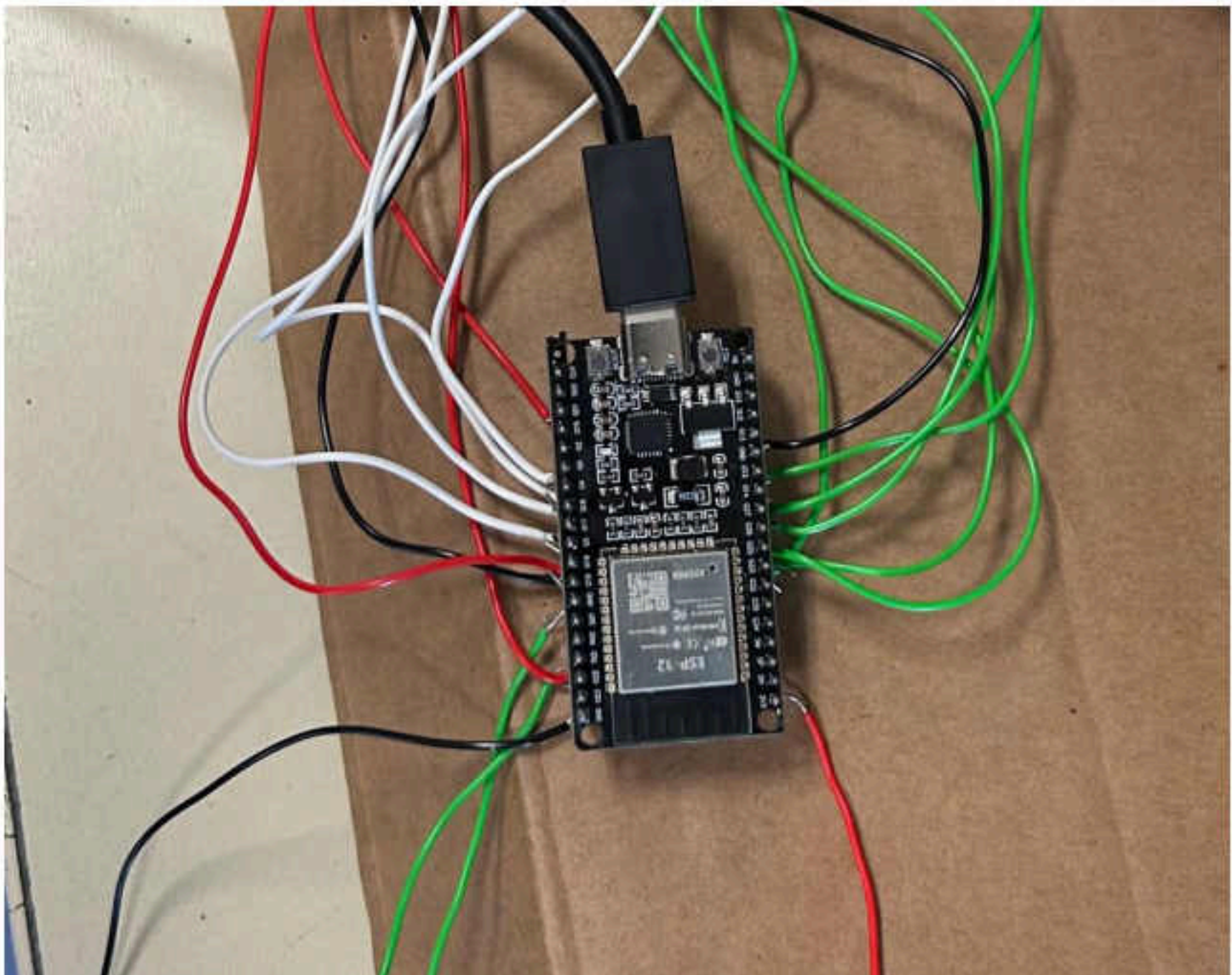
1. **Logique inversée (0V = allumé, 5V = éteint)** : Ce comportement s'explique par une configuration électronique en **anode commune** ou une commande par le pôle négatif (**commutation par la cathode**). La broche du microcontrôleur absorbe le courant (*sink*) au lieu de le fournir (*source*). Lorsque la broche est à 0V (GND), le courant traverse la LED depuis l'alimentation 5V. Lorsqu'elle est à 5V, la différence de potentiel aux bornes de la LED est nulle ($5V - 5V = 0V$), la LED s'éteint.
2. **Modification de la temporisation à 50 000 μ s (50 ms)** :
 - *Comportement visuel* : Le panneau se met à scintiller violemment. On ne voit plus un motif global, mais distinctement chaque ligne s'allume l'une après l'autre de haut en bas.
 - *Phénomène biologique* : En temps normal (avec 500 μ s), c'est la **persistance rétinienne** de l'œil humain qui fusionne ces allumages successifs rapides pour donner l'illusion d'une image fixe et stable.
3. **Le multiplexage partagé et signaux parasites** : Le bus d'adresse A, B, C, D est physiquement câblé en parallèle et **commun** aux deux moitiés du panneau (zone haute et zone basse). Lorsque le décodeur active une adresse (ex: **0000**), il sélectionne simultanément la ligne 1 ET la ligne 17. Si les broches de données R2, G2, B2 de la zone basse sont laissées "en l'air" (non connectées), elles captent des perturbations. Comme la ligne 17 est sélectionnée par le bus d'adresse partagé, ces parasites électriques provoquent des affichages erronés sur la partie basse.

2. EXPLOITATION DE BIBLIOTHÈQUES ET SYNCHRONISATION NTP (Séquence 2)

2.1 Apport du contrôleur DMA

Dans le premier TP, le processeur de l'ESP 32 devait tout exécuter lui-même : envoyer chaque bit, activer les broches et maintenir la ligne allumée. Un simple `delay()` bloquait l'affichage et éteignait le panneau. En Séquence 2, l'utilisation de la bibliothèque [ESP32-HUB75-MatrixPanel-I2S-DMA](#) permet d'exploiter le contrôleur **DMA** (*Direct Memory Access*).

Il s'agit d'un contrôleur matériel indépendant du processeur principal. Son rôle unique est de copier l'image stockée dans la mémoire RAM de l'ESP 32 et de l'envoyer directement aux broches du panneau HUB75 à un rythme ultra-rapide. Ainsi, même si le processeur exécute un `delay(500)`, l'écran reste parfaitement allumé et stable.



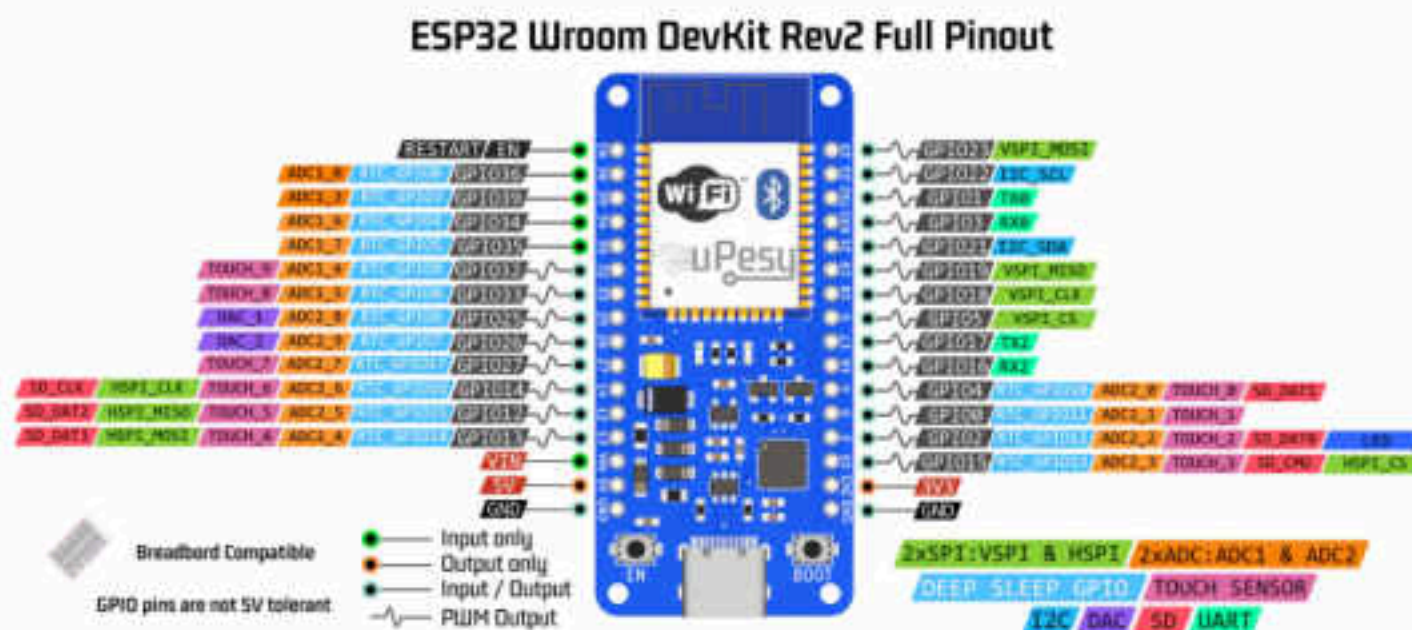
2.2 Réponses aux Questions d'Auto-Contrôle (Séquence 2)

1. Calcul d'encombrement pour "lundi 08 juin" :

- La chaîne de caractères "lundi 08 juin" compte **13 caractères** (espaces compris).
- La police 5x7 intégrée occupe virtuellement **6 pixels de largeur par caractère** (5 pixels + 1 pixel d'espacement).
- *Calcul de la largeur théorique :*
 $13 \times 6 = 78 \text{ pixels}$
- Notre panneau possède une largeur maximale de **64 pixels**. La chaîne mesurant 78 pixels, elle **ne peut pas** s'afficher entièrement de manière statique. Il faudra implémenter un texte défilant pour pouvoir la lire.

2. Le mécanisme NTP (Network Time Protocol) :

- *Absence de pile/RTC :* Dès son démarrage, l'ESP 32 se connecte au réseau Wi-Fi et interroge un serveur de temps (pool.ntp.org) via le protocole NTP. Il récupère instantanément l'heure exacte et l'injecte dans son horloge logicielle interne, s'affranchissant ainsi d'une pile de sauvegarde.
- *Inconvénient majeur :* Si le Wi-Fi de l'établissement tombe en panne, l'ESP32 sera incapable de se synchroniser au démarrage et d'afficher l'heure et la date réelles.





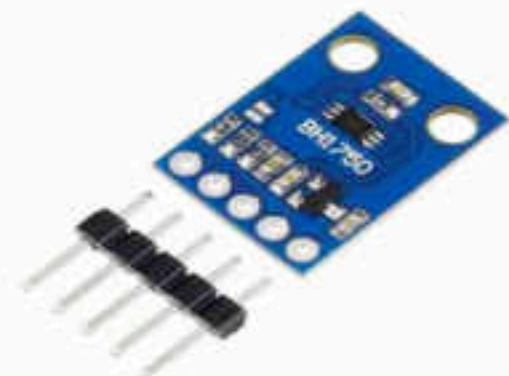
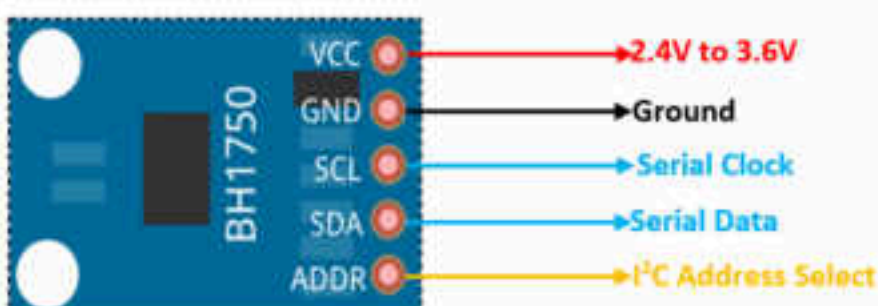
3. **Espacement vertical en mode 4 zones** : Les lignes de texte démarrent aux positions $Y = 0$, $Y = 8$, $Y = 16$ et $Y = 24$. L'espacement vertical exact est de **8 pixels** ($8 - 0 = 8$). Comme la police 5x7 avec son pixel d'espace occupe précisément une hauteur virtuelle de **8 pixels**, chaque texte dispose de son propre corridor strict de 8 pixels de haut, empêchant toute bavure sur la ligne voisine.

3. ACQUISITION I2C ET RÉTROACTION LUMINEUSE (Séquence 3)

3.1 Architecture du Bus I2C et Scanner

Le bus I2C utilise deux lignes de communication : **SDA** (Données) et **SCL** (Horloge). C'est une architecture Maître/Esclave où chaque composant possède une adresse unique sur 7 bits.

- Le scan du bus I2C réalisé en séance permet d'identifier l'adresse matérielle exacte de nos deux composants esclaves (le capteur de température/humidité AHT 20 et le capteur de luminosité BH1750).



BMP280 Pinout



3.2 Algorithme d'adaptation lumineuse automatique

Pour assurer un confort visuel et une éco-conception optimale, la luminosité de la dalle est ajustée de manière dynamique à l'aide de la fonction `display->set Brightness 8()`. Nous traduisons le modèle algorithmique à seuils de la manière suivante:

C++

```
void adapterLuminosite(float luxAmbiant) {  
    int intensité Panneau = 10; // Valeur minimale par défaut (10/255) [cite: 334]  
  
    if (luxAmbiant < 10) { // [cite: 331]  
        intensité Panneau = 10; // Obscurité -> Mode éco [cite: 333, 334]  
    }  
    else if (luxAmbiant > 1000) { // [cite: 335]  
        intensité Panneau = 255; // Pleine lumière -> Intensité max [cite: 337]  
    }  
    else {  
        // Conversion proportionnelle entre 10 Lux et 1000 Lux [cite: 338]  
        intensitéPanneau = map(luxAmbiant, 10, 1000, 10, 255);  
    }  
  
    display->set Brightness 8(intensité Panneau); // Modification de la luminosité du panneau  
}
```

3.3 Justification de l'utilisation de `millis()` face à `delay()`

Dans le code final de notre station météo, l'intégration de la fonction `millis()` est indispensable pour les raisons suivantes:

- Contrairement à la fonction `delay()` qui fige complètement le processeur principal et bloque le programme, `millis()` agit comme un compteur de temps non-bloquant.
- Elle permet de gérer des **fréquences de rafraîchissement asynchrones** : rafraîchir l'affichage graphique toutes les 50 ms pour la fluidité, tout en effectuant l'acquisition des données des capteurs I2C de manière espacée (ex: toutes les 2 secondes).

Resultats finaux

Luminosité basse :



Luminosité haute:



Quand la lumière augmente, l'éclairement en lux augmente et la luminosité du panneau augmente également. À l'inverse, quand la lumière diminue, les lux diminuent et la luminosité du panneau peut diminuer aussi

Conclusion Générale

Cette série de TP de SAE nous a permis de mener à bien la conception d'une **Station Météo Visuelle et Connectée**. Nous avons pu assimiler les bases de la gestion des registres de décalage et du multiplexage temporel bas niveau (Séquence 1) , avant d'optimiser le système via le contrôleur matériel DMA de l'ESP 32 pour l'affichage de données textuelles complexes issues du réseau NTP (Séquence 2). Enfin, l'exploitation du bus I2C couplée à un algorithme de gestion énergétique intelligent (Séquence 3) met en évidence l'intérêt d'une approche d'éco-conception dans les systèmes électroniques embarqués modernes.

